

Corso Python Avanzato – Argomenti Chiave

Serializzazione in JSON

La **serializzazione** consente di convertire oggetti Python in un formato testuale standard (e viceversa) per salvarli su file o trasmetterli in rete. **JSON** (JavaScript Object Notation) è un formato di serializzazione molto diffuso e leggibile da umani, ideale per scambiare dati tra applicazioni. In Python il modulo `json` integrato permette di serializzare strutture dati come dizionari e liste in stringhe JSON, e di effettuare la deserializzazione ricostruendo gli oggetti Python originali.

JSON rappresenta dati con strutture simili ai dizionari (oggetti JSON) e liste (array JSON). È indipendente dal linguaggio e ampiamente usato in **API web** e configurazioni. In Python, funzioni come `json.dumps()` producono una stringa JSON da un oggetto, mentre `json.loads()` fa l'operazione inversa. Si possono anche usare `json.dump()` e `json.load()` per scrivere o leggere direttamente da file JSON.

```
import json
dati = {"nome": "Alice", "eta": 30}
# Serializzazione in JSON (oggetto Python -> stringa JSON)
json_str = json.dumps(dati)
print(json_str)          # Output: '{"nome": "Alice", "eta": 30}'
# Deserializzazione da JSON (stringa JSON -> oggetto Python)
dati2 = json.loads(json_str)
print(dati2["nome"])     # Output: Alice
```

Serializzazione in YAML

YAML (YAML Ain't Markup Language) è un formato di serializzazione testuale orientato alla leggibilità, spesso usato per file di configurazione. A differenza di JSON, YAML permette commenti e una sintassi più flessibile (indentazione per definire strutture). In Python possiamo utilizzare librerie come `PyYAML` (`pip install pyyaml`) per lavorare con YAML in modo simile a JSON.

YAML supporta strutture nidificate (dizionari, liste) in modo intuitivo e spesso è preferito per configurazioni grazie alla sua chiarezza. Con `PyYAML` possiamo convertire oggetti Python in stringhe YAML tramite `yaml.dump()` e fare parsing da YAML a Python con `yaml.safe_load()`. È importante usare `safe_load` per evitare l'esecuzione di codice arbitrario da YAML non fidati.

```
import yaml # Libreria PyYAML
config = {"server": "localhost", "porta": 8080}
# Serializza il dizionario in formato YAML (stringa)
yaml_str = yaml.dump(config)
print(yaml_str)
# Deserializza la stringa YAML in un oggetto Python
```

```
config2 = yaml.safe_load(yaml_str)
print(config2["server"]) # Output: localhost
```

Serializzazione Binaria con Pickle

Oltre ai formati testuali, Python offre **Pickle** per serializzare oggetti in formato binario nativo. Il modulo `pickle` permette di convertire quasi qualsiasi oggetto Python in un byte stream e viceversa (operazioni note come *pickling* e *unpickling*). Questo è utile per salvare strutture complesse su disco o trasferirle tra processi Python.

Il vantaggio di Pickle è che conserva la struttura e il tipo degli oggetti Python senza richiedere conversione manuale. Tuttavia, è un protocollo Python-specifico (non interoperabile con altri linguaggi) e presenta rischi di sicurezza: non bisogna caricare dati Pickle da fonti non fidate, perché potrebbero eseguire codice arbitrario. Funzioni chiave sono `pickle.dumps()` / `loads()` per stringhe di byte e `pickle.dump()` / `load()` per file.

```
import pickle
data = {"x": 42, "lista": [1, 2, 3]}
# Serializza l'oggetto Python in bytes (pickling)
byte_stream = pickle.dumps(data)
# Deserializza il byte stream in un oggetto Python (unpickling)
data2 = pickle.loads(byte_stream)
print(data2["lista"]) # Output: [1, 2, 3]
```

Interazione con Database SQLite (Creazione e Inserimento)

SQLite è un **database relazionale** leggero e integrato, ideale per applicazioni locali o prototipi. Python include il modulo `sqlite3` nella libreria standard, permettendo di creare un database in un file e interagirvi tramite **SQL**. Per iniziare, si stabilisce una connessione (in SQLite può essere anche in-memory) e si ottiene un cursore per eseguire comandi SQL.

Possiamo creare tabelle usando istruzioni `CREATE TABLE` ed inserire dati con `INSERT`. I valori dinamici vanno passati usando placeholder `?` per evitare problemi di SQL injection e gestire correttamente i tipi. Dopo le operazioni di scrittura, è importante fare il commit per salvare le modifiche su disco. Di seguito un esempio di connessione a un database SQLite, creazione di una tabella `utenti` e inserimento di un record.

```
import sqlite3
conn = sqlite3.connect("esempio.db") # Crea/connette a un database SQLite
cur = conn.cursor()
# Crea una tabella 'utenti' con id e nome
cur.execute("CREATE TABLE IF NOT EXISTS utenti (id INTEGER PRIMARY KEY, nome TEXT)")
# Inserisce un nuovo utente in tabella (uso di placeholder ? per il valore)
cur.execute("INSERT INTO utenti (nome) VALUES (?)", ("Alice",))
```

```
conn.commit() # Applica le modifiche al database
print("Inserimento completato")
```

Interazione con Database SQLite (Lettura dei Dati)

Dopo aver inserito dati in un database, tipicamente vogliamo recuperarli. Con SQLite (e SQL in generale) possiamo eseguire query `SELECT` tramite il cursore. I risultati vengono restituiti come tuple di valori Python, una per ogni riga. È possibile usare `cur.fetchall()` per ottenere tutte le righe risultanti o `cur.fetchone()` per una riga alla volta; in alternativa iterare sul cursore stesso.

Nel seguente esempio riapriamo una connessione allo stesso database (oppure riutilizziamo `conn` già aperto) e leggiamo tutti gli utenti inseriti. È buona pratica chiudere la connessione dopo aver finito (`conn.close()`), per liberare risorse. L'esempio mostra come stampare ogni riga ottenuta da una query `SELECT`.

```
import sqlite3
conn = sqlite3.connect("esempio.db")
cur = conn.cursor()
cur.execute("SELECT * FROM utenti")
righe = cur.fetchall()
for r in righe:
    print(r) # Ad esempio: (1, 'Alice')
conn.close() # Chiude la connessione al database
```

Interazione con Database PostgreSQL (Connessione e Inserimento)

PostgreSQL è un database relazionale avanzato client-server. Per connettersi da Python è comune usare librerie come `psycopg2` (`pip install psycopg2`). A differenza di SQLite, occorre specificare parametri di connessione (host, database, utente, password). Una volta connessi, si utilizza un cursore similmente a SQLite per eseguire comandi SQL.

Nel codice seguente si apre una connessione a un server PostgreSQL locale e si crea (se non esiste) una tabella `utenti`. L'inserimento dei dati utilizza placeholder `%s` specifici di `psycopg2` per proteggere da SQL injection. Come con SQLite, è necessario fare `commit` dopo insert/update/delete per rendere permanenti le modifiche. Questo esempio presuppone che esista un database "test" e credenziali valide.

```
import psycopg2
conn = psycopg2.connect(host="localhost", database="test", user="user",
password="pwd")
cur = conn.cursor()
# Crea la tabella utenti se non esiste (id autoincrementale con SERIAL)
cur.execute("CREATE TABLE IF NOT EXISTS utenti (id SERIAL PRIMARY KEY, nome
TEXT)")
# Inserisce un nuovo utente usando placeholder %s per il parametro
cur.execute("INSERT INTO utenti (nome) VALUES (%s)", ("Bob",))
```

```
conn.commit()
print("Inserito utente Bob")
```

Interazione con Database PostgreSQL (Query e Lettura)

Dopo aver inserito dati in PostgreSQL, possiamo interrogarli con `SELECT`. L'uso del cursore rimane simile: eseguiamo la query e poi recuperiamo i risultati. Possiamo iterare sulle righe o utilizzare metodi come `fetchall()`. È buona norma chiudere sia il cursore che la connessione al termine delle operazioni. Inoltre, PostgreSQL supporta transazioni: in caso di errori, si può effettuare un rollback (`conn.rollback()`) se necessario.

Nell'esempio seguente, recuperiamo tutte le righe dalla tabella `utenti` inserite in precedenza e le stampiamo. Notare che stiamo usando la stessa connessione `conn` aperta in precedenza. Per un'applicazione reale, sarebbe opportuno gestire eccezioni (ad esempio con `try/except`) per chiudere connessione/cursore anche in caso di errore.

```
# Esegue una query di selezione
cur.execute("SELECT id, nome FROM utenti")
records = cur.fetchall()
for rec in records:
    print(rec) # Ad esempio: (1, 'Bob')
cur.close()
conn.close()
```

ORM con SQLAlchemy: Definizione del Modello

Interagire con SQL tramite stringhe può essere soggetto a errori e duplicazione. Un **ORM (Object-Relational Mapping)** come **SQLAlchemy** permette di manipolare il database usando oggetti Python. Si definiscono **modelli** (classi Python) che mappano tabelle, e attributi che mappano colonne. SQLAlchemy genera automaticamente le query SQL sottostanti.

Per iniziare con SQLAlchemy, definiamo un motore di database (ad esempio SQLite o PostgreSQL) e una classe base tramite `declarative_base()`. I modelli ereditano da questa base. Nell'esempio definiamo una classe `Utente` con tabella `utenti`, un campo `id` (chiave primaria) e `nome`. Infine, `Base.metadata.create_all(engine)` crea la tabella nel database se non esiste, basandosi sul modello.

```
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy import Column, Integer, String, create_engine

Base = declarative_base()
class Utente(Base):
    __tablename__ = "utenti"
    id = Column(Integer, primary_key=True)
    nome = Column(String)
```

```
engine = create_engine("sqlite:///esempio.db")
Base.metadata.create_all(engine) # Crea la tabella 'utenti' in SQLite
```

ORM con SQLAlchemy: Operazioni CRUD

Una volta definito il modello, SQLAlchemy ORM consente di effettuare operazioni **CRUD** (Create, Read, Update, Delete) in modo oggettuale. Si crea una sessione di database tramite `sessionmaker` e la si usa per aggiungere, modificare o queryare oggetti. Gli oggetti restituiti dalla query sono istanze delle classi modello definite.

Nell'esempio, apriamo una nuova sessione e creiamo un oggetto `Utente` (con nome "Carlo"), aggiungendolo alla sessione e facendo commit per salvare sul DB. Poi eseguiamo una query su tutti gli utenti presenti: `session.query(Utente).all()` restituisce una lista di oggetti `Utente`. Iterandoli, accediamo ai campi come attributi Python. Infine, chiudiamo la sessione. Questo approccio elimina la necessità di scrivere SQL manualmente per molte operazioni comuni.

```
from sqlalchemy.orm import sessionmaker

Session = sessionmaker(bind=engine)
session = Session()
# Crea un nuovo utente e lo aggiunge al database
nuovo = Utente(nome="Carlo")
session.add(nuovo)
session.commit()
# Lettura: query di tutti gli utenti
for utente in session.query(Utente).all():
    print(utente.id, utente.nome) # Es: 1 Carlo
session.close()
```